

Glea Nexo — roadmap unificado de ejercicios de mejora y aprendizaje aplicado

Fecha: 2026-05-31

Versión: v2 — limpieza de referencias, sprint inicial, fichas rápidas, tamaño, prioridad, dependencias y límites de alcance.

Ámbito: unificación y reorganización de los ejercicios base y avanzados en un único fichero operativo.

Objetivo: priorizar el aprendizaje profundo mientras Glea Nexo evoluciona hacia un producto robusto, escalable, fiable y mantenible.

Referencias principales del repo

Antes de ejecutar ejercicios, conviene tener a mano estas fuentes de verdad:

- `agents.md` : guía operativa para agentes y comandos rápidos.
 - `README.md` : visión general, API disponible y quickstart.
 - `roadmap.md` : roadmap funcional amplio del proyecto.
 - `session-2026-05-30.md` : sesión de corrección de telemetría y parametrización edge.
 - `docs/iterations/` : histórico de iteraciones implementadas.
 - `docs/runbook/` : procedimientos operativos existentes.
 - `infra/compose/` : Compose edge/platform/rpi.
 - `edge/nodered/flows/flows.json` : flow versionado de Node-RED.
 - `backend/src/main/java/com/glea/nexo/**` : backend real.
 - `frontend/src/app/**` : frontend real.
-

Cómo usar este roadmap

Este documento no debe usarse como una lista de tareas sueltas. Cada ejercicio debe cerrarse como una mini iteración profesional con alcance claro, evidencia técnica, documentación versionada y una defensa breve de la decisión tomada.

Ciclo recomendado de trabajo:

```
Issue → rama → implementación/prueba → evidencia → documentación → defensa técnica → merge
```

Cada ejercicio solo cuenta si termina con:

- evidencia real en código, logs, SQL, tests, capturas, Swagger, Node-RED, MQTT o runbook;
 - criterio de `done` explícito;
 - limitaciones conocidas;
 - explicación breve en formato `problema → decisión → tradeoff → riesgo`.
-

Principios de priorización

Este roadmap prioriza primero la **realidad operativa** del sistema, después la **claridad de contratos y semántica**, luego la **robustez técnica**, más tarde la **capacidad de producto visible**, y finalmente la **profesionalización** del proyecto.

La lógica es simple:

1. Si el edge no funciona de verdad, el resto es maquillaje.
2. Si el tiempo, los errores y los límites no están claros, el backend será frágil.
3. Si no hay seguridad, gobernanza de datos y operación reproducible, no hay producto serio.
4. Si no hay demo, UX y narrativa técnica, el valor del sistema no se percibe.

Estructura común de cada ejercicio

Cada ejercicio debe responder al menos a estas preguntas:

- ¿Qué resuelve?
- ¿Qué aprendizaje exige?
- ¿Qué documentación o código revisar antes?
- ¿Qué hitos concretos tiene?
- ¿Cómo se comprueba?
- ¿Cómo se documenta al cerrar?

Cuando un ejercicio original era demasiado grande, aquí se ha desglosado en mini ejercicios para reducir riesgo, mejorar foco y facilitar el aprendizaje incremental.

Orden maestro recomendado

```
Bloque 1 → Realidad edge y flujo base
Bloque 2 → Semántica temporal e idempotencia
Bloque 3 → Contratos API, errores y límites
Bloque 4 → Observabilidad y rendimiento
Bloque 5 → Datos, esquema y gobierno técnico
Bloque 6 → Seguridad incremental
Bloque 7 → Robustez edge/offline
Bloque 8 → Producto visible y frontend
Bloque 9 → Alertas y operación
Bloque 10 → Actuadores y control cerrado
Bloque 11 → Refactor arquitectónico y mantenibilidad
Bloque 12 → Profesionalización y entrega
```

Convención de planificación por ejercicio

Cada ejercicio incluye una **ficha rápida** para poder convertirlo en issue real del repo sin volver a pensar el alcance desde cero.

- **Tipo:** aprendizaje, producto, hardening, operación, arquitectura o portfolio.
- **Área:** subsistema principal afectado.
- **Prioridad:** **P0** crítico, **P1** importante, **P2** mejora planificable.
- **Tamaño:** **S** pequeño, **M** medio, **L** grande.
- **Depende de:** ejercicios que conviene cerrar antes.
- **Desbloquea:** ejercicios que se vuelven más seguros o útiles después.
- **No entra:** límite explícito para evitar que el ejercicio crezca sin control.

Sprint inicial recomendado

Objetivo: validar la verdad operativa mínima de Glea Nexa antes de invertir más esfuerzo en UI, seguridad avanzada o actuadores.

Ejercicios incluidos:

1 → 2 → 3 → 4 → 6

Criterio de cierre del sprint:

- una lectura entra desde MQTT hasta PostgreSQL;
- una caída edge → backend deja datos pendientes en el outbox o demuestra claramente que esa capacidad está ausente;
- al restaurar conectividad, el replay se valida o se documenta el gap exacto;
- un `messageId` reenviado no duplica datos funcionales;
- queda un runbook reproducible con comandos, SQL, logs y limitaciones conocidas.

No entra en este sprint:

- implementar WebSockets;
- rediseñar el frontend;
- cerrar JWT completo;
- implementar actuadores reales;
- migrar la base de datos a una solución time-series.

Bloque 1 — Realidad edge y flujo base

Ejercicio 1 — Preparar entorno edge real

Ficha rápida

Campo	Valor
Tipo	operación + aprendizaje edge
Área	Edge / Docker Compose / Node-RED / Python
Prioridad	P0
Tamaño	M
Depende de	Ninguno
Desbloquea	2, 3, 20, 26
No entra	No implementar nuevas features; no tocar seguridad completa; no rediseñar flows salvo configuración mínima.

¿Qué resuelve?

Asegura que la validación del sistema se hace sobre un entorno real y no sobre supuestos.

Mini ejercicios

1. Identificar si el edge real será PC o Raspberry Pi.
2. Confirmar IP del backend accesible desde el edge.
3. Verificar `BACKEND_URL` en Compose y Node-RED.
4. Confirmar que el simulador Python no sigue en `sleep`.
5. Levantar el stack completo y validar healths básicos.
6. Verificar que los Dockerfiles reflejan puertos reales (backend `EXPOSE 8081`, frontend con build estático para producción).

Done

- Stack levantado.
- Simulador listo o limitación documentada.
- Backend accesible desde edge.
- Runbook de arranque reproducible.

Ejercicio 2 — Validar ingest normal end-to-end

Ficha rápida

Campo	Valor
Tipo	validación funcional
Área	MQTT → Node-RED → Backend → PostgreSQL
Prioridad	P0
Tamaño	M
Depende de	1
Desbloquea	3, 4, 6, 11, 13
No entra	No optimizar rendimiento todavía; no añadir dashboard; no cambiar contratos si solo estás validando.

¿Qué resuelve?

Demuestra que una lectura real entra por MQTT y termina persistida correctamente en PostgreSQL.

Mini ejercicios

1. Publicar una lectura con `messageId` conocido.
2. Verla pasar por Node-RED.
3. Confirmar si toca SQLite/outbox o va directa al backend.
4. Verificar `IngestEvent` y `TelemetryReading` en PostgreSQL.
5. Consultar `latest` y `readings` para confirmar visibilidad.

Done

- Una lectura trazada de extremo a extremo.
- Evidencia en logs + SQL + endpoint.

Ejercicio 3 — Offline y replay controlado

Ficha rápida

Campo	Valor
Tipo	resiliencia + operación
Área	Edge offline / SQLite outbox / replay
Prioridad	P0
Tamaño	L
Depende de	1, 2
Desbloquea	6, 23, 24, 25, 34
No entra	No implementar una cola nueva desde cero; no meter Kafka/RabbitMQ; no prometer offline real sin evidencia.

¿Qué resuelve?

Valida si la promesa edge-first existe de verdad bajo caída de conectividad.

Mini ejercicios

1. Cortar conectividad edge → backend manteniendo MQTT/Node-RED activos.
2. Generar telemetría durante la caída.
3. Verificar acumulación en SQLite/outbox.
4. Restaurar conectividad.
5. Verificar replay y ausencia de pérdidas.
6. Clasificar el resultado como **REAL** , **PARCIAL** o **AUSENTE** .

Done

- Veredicto técnico con evidencia.
 - Runbook de prueba offline/replay.
-

Bloque 2 — Semántica temporal e idempotencia

Ejercicio 4 — Trazabilidad temporal completa

Ficha rápida

Campo	Valor
Tipo	diseño de datos + trazabilidad
Área	Timestamps / IngestEvent / TelemetryReading
Prioridad	P0
Tamaño	M
Depende de	2
Desbloquea	5, 33
No entra	No cambiar todos los nombres de campos sin migración; no mezclar hora local y UTC.

¿Qué resuelve?

Aclara qué significa cada timestamp del sistema y evita confundir tiempo de evento con tiempo de ingest.

Mini ejercicios

1. Elegir una lectura con timestamp explícito.
2. Trazarla por MQTT, Node-RED, SQLite, `IngestEvent` y `TelemetryReading`.
3. Identificar campos `event time`, `received time`, `processed time`, `persisted time`.
4. Detectar ambigüedades o sobrescrituras.
5. Elegir el timestamp canónico de negocio.

Done

- Documento `time-semantics`.
- Regla canónica escrita.

Ejercicio 5 — Endurecer contrato temporal

Ficha rápida

Campo	Valor
Tipo	calidad backend + contrato API
Área	Validación temporal / Swagger / tests
Prioridad	P1
Tamaño	M
Depende de	4
Desbloquea	10, 28, 33
No entra	No abrir nuevos endpoints; no introducir reglas de negocio complejas.

¿Qué resuelve?

Convierte la semántica temporal en validaciones y reglas verificables.

Mini ejercicios

1. Definir formatos aceptados y obligatoriedad de campos temporales.
2. Decidir política para timestamps futuros, nulos o ambiguos.
3. Validar `from ≤ to` y rango máximo de consulta.
4. Añadir tests de fechas inválidas.
5. Actualizar Swagger/OpenAPI con ejemplos válidos e inválidos.

Done

- Tests verdes.
- Contrato temporal documentado.
- Casos inválidos devuelven error controlado.

Ejercicio 6 — Validar deduplicación real

Ficha rápida

Campo	Valor
Tipo	fiabilidad de ingest
Área	Idempotencia / constraints / replay
Prioridad	P0
Tamaño	M
Depende de	2, 3
Desbloquea	11, 23, 24, 41
No entra	No asumir dedupe por logs; no tocar constraints sin migración o test.

¿Qué resuelve?

Comprueba si la idempotencia aguanta reenvíos y replay sin crear duplicados funcionales.

Mini ejercicios

1. Identificar claves y constraints reales de deduplicación.
2. Enviar el mismo batch dos veces.
3. Enviar mismo `messageId` con payload distinto y analizar el comportamiento.
4. Validar SQL sin duplicados.
5. Clasificar la garantía como `REAL` , `PARCIAL` o `AUSENTE` .

Done

- Evidencia en SQL.
- Riesgo residual escrito.

Cierre del Bloque 2

El Bloque 2 queda formalmente cerrado. La semántica temporal del sistema quedó aclarada y documentada, el contrato temporal fue endurecido y verificado con pruebas, y la idempotencia de ingest fue validada con evidencia real en API y SQL.

La conclusión de cierre es que la deduplicación actual debe clasificarse como **PARCIAL**: evita duplicados funcionales cuando se reutiliza la misma clave técnica, pero no detecta conflicto semántico si un productor reutiliza un `messageId` con payload distinto. El riesgo residual principal queda identificado y aceptado para esta fase, con seguimiento pendiente en preparación de demo técnica y futuras tareas de hardening.

Bloque 3 — Contratos API, errores y límites

Ejercicio 7 — OpenAPI como contrato vivo

Ficha rápida

Campo	Valor
Tipo	contrato API
Área	OpenAPI / Swagger / DTOs
Prioridad	P1
Tamaño	M
Depende de	2, 5
Desbloquea	8, 10, 21, 29
No entra	No cambiar comportamiento solo por documentar; no duplicar documentación contradictoria.

¿Qué resuelve?

Evita que la API dependa de memoria tácita o lectura directa del código.

Mini ejercicios

1. Inventariar endpoints existentes.
2. Documentar parámetros críticos (`X-Org-Code` , filtros, paginación, fechas).
3. Añadir ejemplos reales de request/response.
4. Verificar que Swagger se entiende sin abrir código.

Done

- Swagger claro y usable.
- Docs API actualizadas.

Ejercicio 8 — Matriz de status codes

Ficha rápida

Campo	Valor
Tipo	calidad REST
Área	Status codes / errores / tests
Prioridad	P1
Tamaño	M
Depende de	7
Desbloquea	9, 19, 21
No entra	No devolver 500 para casos controlables; no crear formatos de error múltiples.

¿Qué resuelve?

Evita que casos controlables terminen en `500`.

Mini ejercicios

1. Crear tabla `endpoint → caso → status esperado`.
2. Probar validación, not found, conflicto y error inesperado.
3. Alinear el cuerpo de error con el formato estándar.
4. Actualizar documentación con ejemplos de error.

Done

- Matriz de errores versionada.
- Al menos 3 tests nuevos.

Ejercicio 9 — ControllerAdvice serio

Ficha rápida

Campo	Valor
Tipo	hardening backend
Área	@ControllerAdvice / GlobalExceptionHandler
Prioridad	P1
Tamaño	M
Depende de	8
Desbloquea	21, 29
No entra	No capturar excepciones ocultando bugs; no eliminar correlationId.

¿Qué resuelve?

Centraliza errores y reduce respuestas inconsistentes entre controladores.

Mini ejercicios

1. Auditar excepciones ya cubiertas y no cubiertas.
2. Definir DTO de error único.
3. Añadir mappings explícitos para JSON inválido, Bean Validation, not found, conflicto y access denied.
4. Asegurar logging correcto con `correlationId`.

Done

- Respuestas coherentes en toda la API.
 - Tests representativos de error.
-

Ejercicio 10 — Límites y paginación defensiva

Ficha rápida

Campo	Valor
Tipo	escalabilidad defensiva
Área	Paginación / filtros / rangos temporales
Prioridad	P1
Tamaño	M
Depende de	5, 7
Desbloquea	28, 41
No entra	No resolver agregaciones históricas avanzadas; no cambiar UX todavía.

¿Qué resuelve?

Protege al sistema frente a consultas peligrosas o históricas excesivas.

Mini ejercicios

1. Detectar endpoints sin paginación o sin límites razonables.
2. Definir tamaño máximo de página.
3. Definir rango temporal máximo sin agregación.
4. Validar `page`, `size`, `from`, `to` y orden esperado.
5. Añadir tests y documentación de límites.

Done

- Límites documentados.
 - Peticiones abusivas rechazadas o normalizadas.
-

Bloque 4 — Observabilidad y rendimiento

Ejercicio 11 — Instrumentación mínima end-to-end

Ficha rápida

Campo	Valor
Tipo	observabilidad técnica
Área	MDC / correlationId / logs / Node-RED
Prioridad	P0
Tamaño	M
Depende de	2, 6
Desbloquea	12, 14, 34, 41
No entra	No montar Prometheus todavía; no introducir tracing distribuido complejo.

¿Qué resuelve?

Hace trazable una lectura completa sin adivinar a partir de logs dispersos.

Mini ejercicios

1. Confirmar soporte de `X-Correlation-Id`.
2. Propagar correlación desde Node-RED al backend.
3. Incluir `messageId` en logs relevantes.
4. Añadir duración de procesamiento por batch o ítem.
5. Validar el trazado completo con una lectura real.

Done

- Una lectura se sigue de extremo a extremo con logs.

Ejercicio 12 — Presupuesto de latencia

Ficha rápida

Campo	Valor
Tipo	rendimiento aplicado
Área	Latencia end-to-end
Prioridad	P1
Tamaño	M
Depende de	11
Desbloquea	13, 30
No entra	No micro-optimizar sin datos; no cambiar arquitectura por una medición aislada.

¿Qué resuelve?

Permite optimizar con criterio en lugar de intuición.

Mini ejercicios

1. Dibujar el flujo completo con tramos medibles.
2. Definir presupuesto objetivo por tramo.
3. Medir ingest individual.
4. Medir lote pequeño y lote mayor.
5. Separar latencia de ingest de latencia visible en frontend.
6. Decidir qué no se optimiza todavía.

Done

- Tabla de latencia por tramo.
- 3 decisiones derivadas del análisis.

Ejercicio 13 — Radiografía SQL de consultas de observabilidad

Ficha rápida

Campo	Valor
Tipo	rendimiento SQL
Área	PostgreSQL / índices / queries telemetry
Prioridad	P1
Tamaño	M
Depende de	2, 11, 12
Desbloquea	15, 28
No entra	No añadir índices al azar; no usar datos irreales para conclusiones definitivas.

¿Qué resuelve?

Permite saber si `readings` y `latest` son sostenibles antes de crecer en datos y producto.

Mini ejercicios

1. Localizar SQL real de ambos endpoints.
2. Revisar filtros y ordenaciones.
3. Inventariar índices existentes.
4. Ejecutar `EXPLAIN / EXPLAIN ANALYZE` con datos representativos.
5. Decidir índice nuevo o deuda consciente.

Done

- Análisis SQL documentado.
- Propuesta de índices o justificación para no tocar aún.

Ejercicio 14 — Métricas mínimas con Actuator/Prometheus

Ficha rápida

Campo	Valor
Tipo	observabilidad operativa
Área	Spring Actuator / métricas / Prometheus
Prioridad	P2
Tamaño	L
Depende de	11
Desbloquea	19, 24, 41
No entra	No montar una plataforma enterprise de observabilidad; no bloquear el roadmap si queda como diseño.

¿Qué resuelve?

Complementa los logs con métricas de comportamiento y salud.

Mini ejercicios

1. Revisar qué expone Actuator hoy.
2. Exponer métricas útiles de ingest y error rate.
3. Añadir scrape básico con Prometheus.
4. Crear un dashboard mínimo.
5. Definir una alerta simple.

Done

- Dashboard funcional con métricas reales.
- Una alerta mínima documentada.

Bloque 5 — Datos, esquema y gobierno técnico

Ejercicio 15 — Gobierno del esquema: de `ddl-auto=update` a migraciones controladas

Ficha rápida

Campo	Valor
Tipo	gobierno técnico
Área	Flyway / Hibernate / PostgreSQL
Prioridad	P1
Tamaño	L
Depende de	13
Desbloquea	16, 17, 18, 44
No entra	No migrar a TimescaleDB; no tocar datos productivos sin backup; no mezclar <code>ddl-auto=update</code> con decisiones finales.

¿Qué resuelve?

Reduce deriva de esquema y evita errores como los de naming entre Hibernate y la base de datos.

Mini ejercicios

1. Auditar configuración actual de Flyway y Hibernate.
2. Comparar entidades y tablas reales.
3. Inventariar qué depende hoy de `ddl-auto=update`.
4. Crear migraciones para cerrar gaps.
5. Cambiar entorno controlado a `ddl-auto=validate`.
6. Verificar arranque y tests.

Done

- Esquema gobernado por migraciones.
- Documento de gobierno de esquema.

Ejercicio 16 — Backup y restore de PostgreSQL

Ficha rápida

Campo	Valor
Tipo	operación de datos
Área	PostgreSQL backup/restore
Prioridad	P1
Tamaño	M
Depende de	15
Desbloquea	44
No entra	No diseñar alta disponibilidad; no automatizar cloud backups todavía.

¿Qué resuelve?

Asegura recuperación básica ante pérdida de datos o rotura del entorno.

Mini ejercicios

1. Cargar datos mínimos representativos.
2. Hacer backup con `pg_dump`.
3. Restaurar sobre base limpia o temporal.
4. Validar conteos y muestras.
5. Arrancar backend contra la base restaurada.

Done

- Runbook reproducible de backup/restore.

Ejercicio 17 — Política de retención de telemetría

Ficha rápida

Campo	Valor
Tipo	arquitectura de datos
Área	Retención / almacenamiento / costes
Prioridad	P2
Tamaño	M
Depende de	13, 15
Desbloquea	18, 28
No entra	No implementar archivado físico todavía; no sobredimensionar para millones de sensores.

¿Qué resuelve?

Anticipa crecimiento de datos y evita decisiones tardías de escalabilidad.

Mini ejercicios

1. Calcular lecturas/día para 10, 100 y 1000 sensores.
2. Estimar crecimiento mensual y anual.
3. Definir retención MVP de crudo, agregados y archivo.
4. Identificar endpoints afectados.
5. Escribir decisión explícita de qué no se implementa todavía.

Done

- Política de retención versionada.
 - Supuestos y riesgos documentados.
-

Ejercicio 18 — ADR de escalabilidad temporal

Ficha rápida

Campo	Valor
Tipo	arquitectura + ADR
Área	Escalabilidad temporal / PostgreSQL / time-series
Prioridad	P2
Tamaño	S
Depende de	17
Desbloquea	40
No entra	No migrar tecnología; solo decidir criterios futuros.

¿Qué resuelve?

Te obliga a pensar si PostgreSQL actual basta o cuándo necesitarás particionado u otra estrategia.

Mini ejercicios

1. Revisar volumen esperado según política de retención.
2. Analizar si basta PostgreSQL normal, particionado o solución time-series futura.
3. Redactar ADR con contexto, alternativas y consecuencias.

Done

- ADR aprobable y defendible oralmente.
-

Bloque 6 — Seguridad incremental

Ejercicio 19 — Revisión OWASP corta y priorizada

Ficha rápida

Campo	Valor
Tipo	seguridad defensiva
Área	OWASP / superficie expuesta
Prioridad	P1
Tamaño	M
Depende de	7, 8, 11
Desbloquea	20, 21, 22
No entra	No implementar todas las mitigaciones; priorizar top 3 reales.

¿Qué resuelve?

Introduce seguridad como parte del criterio de ingeniería y no como añadido tardío.

Mini ejercicios

1. Auditar superficie expuesta: backend, Swagger, Node-RED (verificar `adminAuth` habilitado o proponerlo), Mosquitto, PostgreSQL.
2. Revisar hardening de contenedores: verificar que los Dockerfiles no corren como root, evaluar `read_only`, `cap_drop`, secretos en compose, y puertos expuestos correctos.
3. Revisar inputs, CORS, errores y secretos.
4. Priorizar top 3 riesgos.
5. Proponer top 3 mitigaciones MVP.

Done

- Tabla de riesgos priorizada.

Ejercicio 20 — Cerrar MQTT anónimo

Ficha rápida

Campo	Valor
Tipo	hardening edge
Área	Mosquitto auth / ACL
Prioridad	P1
Tamaño	M
Depende de	1, 19
Desbloquea	21, 26
No entra	No montar TLS/mTLS completo si no toca; no romper simulador/Node-RED sin plan de credenciales.

¿Qué resuelve?

Evita que cualquier cliente de red publique o escuche topics sin control.

Mini ejercicios

1. Crear usuario y contraseña de broker.
2. Desactivar `allow_anonymous`.
3. Crear ACL mínima.
4. Ajustar Node-RED y simulador.
5. Probar publicación autorizada y no autorizada.

Done

- MQTT autenticado funcionando.
- Prueba negativa documentada.

Ejercicio 21 — JWT mínimo para API

Ficha rápida

Campo	Valor
Tipo	hardening backend
Área	Spring Security / JWT / RBAC
Prioridad	P1
Tamaño	L
Depende de	8, 19
Desbloquea	22, 36, 37
No entra	No implementar OAuth2/OIDC empresarial; no proteger health de forma que rompa checks locales.

¿Qué resuelve?

Cierra endpoints que hoy están abiertos por defecto.

Mini ejercicios

1. Definir endpoints públicos y protegidos.
2. Definir roles mínimos.
3. Implementar validación JWT o MVP equivalente.
4. Proteger endpoints.
5. Crear tests `401/403`.

Done

- Endpoints protegidos.
- Matriz rol/acción documentada.

Ejercicio 22 — Auditoría de acciones críticas

Ficha rápida

Campo	Valor
Tipo	trazabilidad de negocio
Área	AuditLog / acciones críticas
Prioridad	P1
Tamaño	M
Depende de	21
Desbloquea	36, 37, 44
No entra	No auditar cada lectura de telemetría; centrar acciones humanas y críticas.

¿Qué resuelve?

Añade trazabilidad de negocio para cambios sensibles y futuros actuadores.

Mini ejercicios

1. Diseñar entidad `AuditLog`.
2. Crear migración.
3. Auditar una acción simple de inventario.
4. Propagar `correlationId`.
5. Exponer consulta paginada.

Done

- Al menos una acción auditada de forma real.

Bloque 7 — Robustez edge y operación offline

Ejercicio 23 — Contrato formal del outbox

Ficha rápida

Campo	Valor
Tipo	resiliencia edge
Área	SQLite outbox / estados / replay
Prioridad	P0
Tamaño	M
Depende de	3, 6
Desbloquea	24, 25
No entra	No sustituir Node-RED; no diseñar cola distribuida; no duplicar tablas sin necesidad.

¿Qué resuelve?

Evita que SQLite sea solo una cola informal sin semántica de estados.

Mini ejercicios

1. Revisar schema actual.
2. Definir estados (`PENDING` , `SENDING` , `SENT` , `FAILED` , `DEAD_LETTER` o equivalentes).
3. Definir campos mínimos: `tries` , `last_error` , `next_attempt_at` , `sent_at` .
4. Comparar con flujo real.
5. Ajustar implementación o documentar gap.

Done

- Diagrama de estados del outbox.
- Queries operativas de inspección.

Ejercicio 24 — Backoff real en Node-RED

Ficha rápida

Campo	Valor
Tipo	resiliencia edge
Área	Node-RED retry / backoff
Prioridad	P0
Tamaño	L
Depende de	23
Desbloquea	3, 34, 41
No entra	No hacer reintentos infinitos; no saturar backend; no perder mensajes por marcar SENT antes de tiempo.

¿Qué resuelve?

Evita retry agresivo y protege tanto al edge como al backend durante fallos prolongados.

Mini ejercicios

1. Localizar el POST actual al backend.
2. Diseñar política de retry y backoff.
3. Persistir `tries` y `next_attempt_at` .
4. Probar con backend caído.
5. Validar recuperación tras volver el backend.

Done

- Política de retry demostrada con evidencia.

Ejercicio 25 — Dead Letter Queue local

Ficha rápida

Campo	Valor
Tipo	resiliencia edge
Área	DLQ local / payloads inválidos
Prioridad	P1
Tamaño	M
Depende de	23
Desbloquea	34, 41
No entra	No resolver validación semántica completa; separar errores permanentes de transitorios.

¿Qué resuelve?

Evita que mensajes imposibles bloqueen el procesamiento de mensajes válidos.

Mini ejercicios

1. Clasificar errores permanentes vs transitorios.
2. Crear estado o tabla DLQ.
3. Guardar payload y error.
4. Verificar que mensajes sanos siguen fluyendo.
5. Definir procedimiento manual de revisión o reinyectado.

Done

- DLQ operable y documentada.

Ejercicio 26 — LWT y estado de gateway

Ficha rápida

Campo	Valor
Tipo	estado operativo
Área	MQTT LWT / gateway status
Prioridad	P1
Tamaño	M
Depende de	1, 20
Desbloquea	33, 35
No entra	No confundir LWT con telemetría; no asumir backend offline por sensor offline.

¿Qué resuelve?

Distingue mejor entre sensor caído, gateway caído, backend caído y ausencia temporal de datos.

Mini ejercicios

1. Revisar LWT actual del simulador.
2. Definir semántica de `online/offline`.
3. Probar arranque limpio y parada abrupta.
4. Decidir si el backend debe persistir esos estados.
5. Integrar la semántica en snapshot o frontend cuando exista.

Done

- Estados de gateway/sensor definidos y probados.

Bloque 8 — Producto visible y frontend

Ejercicio 27 — Snapshot por zona

Ficha rápida

Campo	Valor
Tipo	producto backend
Área	Snapshot por zona
Prioridad	P1
Tamaño	M
Depende de	10, 13, 33
Desbloquea	29, 30, 35
No entra	No construir dashboard completo; no meter reglas avanzadas de alertas.

¿Qué resuelve?

Cierra una necesidad operativa clara para producto y frontend.

Mini ejercicios

1. Definir qué significa snapshot por zona.
2. Elegir campos mínimos de respuesta.
3. Definir diferencia frente a `latest`.
4. Diseñar contrato.
5. Implementar endpoint o especificación cerrada con test de aceptación.

Done

- Contrato claro y defendible.

Ejercicio 28 — Histórico/serie para frontend

Ficha rápida

Campo	Valor
Tipo	producto backend
Área	Histórico / series / contrato frontend
Prioridad	P1
Tamaño	M
Depende de	10, 13
Desbloquea	31
No entra	No duplicar endpoints si readings ya sirve; no devolver datasets sin límite.

¿Qué resuelve?

Decide si el endpoint actual sirve para gráficos o si hace falta uno más específico.

Mini ejercicios

1. Revisar respuesta actual de `readings`.
2. Definir lo que realmente necesita el frontend para gráficos.
3. Decidir si crear `series` o reutilizar `readings`.
4. Establecer límites y filtros.
5. Probar el contrato con una consulta reproducible.

Done

- Contrato de serie documentado.

Ejercicio 29 — Servicio Angular tipado de telemetría

Ficha rápida

Campo	Valor
Tipo	frontend arquitectura
Área	Angular service / DTOs / HTTP
Prioridad	P1
Tamaño	M
Depende de	7, 27, 28
Desbloquea	30, 31, 32
No entra	No crear arquitectura de módulos compleja; no mezclar HTTP en templates/componentes.

¿Qué resuelve?

Reduce acoplamiento entre UI y construcción manual de requests.

Mini ejercicios

1. Previo: actualizar `app.component.spec.ts` para que refleje el componente real y los tests pasen (hoy están desactualizados respecto al template y las signals).
2. Crear interfaces DTO.
3. Crear `TelemetryApiService`.
4. Centralizar `apiBaseUrl`.
5. Mover llamadas HTTP fuera del componente.
6. Añadir estados loading/error/empty.
7. Verificar que los tests unitarios del componente y del nuevo servicio pasan.

Done

- Servicio frontend reutilizable y tipado.

Ejercicio 30 — Polling cada 5 segundos con cancelación limpia

Ficha rápida

Campo	Valor
Tipo	frontend realtime pragmático
Área	Angular / RxJS polling
Prioridad	P1
Tamaño	M
Depende de	29
Desbloquea	32, 35
No entra	No implementar WebSockets/SSE; no hacer polling sin cancelación.

¿Qué resuelve?

Permite sensación de tiempo real sin introducir complejidad prematura.

Mini ejercicios

1. Elegir endpoint de polling.
2. Montar flujo RxJS con limpieza correcta.
3. Evitar timers duplicados.
4. Mostrar estado stale/offline si backend falla.
5. Comprobar recuperación.

Done

- Polling estable y sin fugas.

Ejercicio 31 — Gráfico temporal útil para demo y operación

Ficha rápida

Campo	Valor
Tipo	frontend visualización
Área	Chart.js / histórico
Prioridad	P1
Tamaño	M
Depende de	28, 29
Desbloquea	32, 42
No entra	No hacer dashboard BI completo; no añadir librerías pesadas sin justificar.

¿Qué resuelve?

Transforma histórico bruto en una visualización comprensible.

Mini ejercicios

1. Integrar Chart.js.
2. Cargar histórico.
3. Implementar rangos 1h / 24h / 7d.
4. Gestionar estados sin datos y errores.
5. Validar en móvil.

Done

- Gráfica funcional y usable.

Ejercicio 32 — UX móvil de campo

Ficha rápida

Campo	Valor
Tipo	producto UX
Área	Angular mobile-first
Prioridad	P1
Tamaño	M
Depende de	29, 30, 31
Desbloquea	42
No entra	No rediseñar marca completa; centrar operación en campo y estados críticos.

¿Qué resuelve?

Adapta la UI a uso real: móvil, poca atención, mala conexión y decisión rápida.

Mini ejercicios

1. Revisar la UI en viewport móvil.
2. Rediseñar cards críticas.
3. Asegurar contraste y targets táctiles.
4. Hacer visibles estados OK/alerta/offline/stale.
5. Validar demo de 5 minutos sin fricción.

Done

- UI móvil útil, no solo “responsive”.

Bloque 9 — Alertas y operación

Ejercicio 33 — Semántica de stale/offline

Ficha rápida

Campo	Valor
Tipo	semántica operativa
Área	stale/offline/unknown
Prioridad	P1
Tamaño	M
Depende de	4, 26
Desbloquea	27, 35
No entra	No confundir ausencia de datos con fallo de sensor; no crear demasiados estados.

¿Qué resuelve?

Distingue ausencia de datos, retraso y caída real.

Mini ejercicios

1. Definir `online`, `offline`, `stale`, `unknown`.
2. Decidir si el estado sale de `lastSeenAt`, LWT o ambos.
3. Probar casos borde.
4. Definir umbral stale.
5. Añadir `statusReason` o equivalente.
6. Verificar que `IngestItemProcessor` actualiza `Device.lastSeenAt` además de `Sensor.lastSeenAt`. Si no lo hace, corregirlo.
7. Probar alerta `STALE_DEVICE` con un device que solo recibe telemetría vía ingest.

Done

- Semántica estable y explicable.
- `Device.lastSeenAt` consistente con la última telemetría recibida.
- Alerta stale funciona correctamente tras una corrección o deuda documentada.

Ejercicio 34 — Modos de fallo esperables

Ficha rápida

Campo	Valor
Tipo	operación + resiliencia
Área	Failure modes / runbooks
Prioridad	P1
Tamaño	M
Depende de	3, 11, 33
Desbloquea	35, 41
No entra	No cubrir todos los fallos posibles; probar al menos 3 representativos.

¿Qué resuelve?

Convierte los fallos en comportamiento esperado y observable.

Mini ejercicios

1. Listar fallos probables: API, PostgreSQL, MQTT, Node-RED, replay parcial, timestamps corruptos.
2. Definir señal observable para cada uno.
3. Probar al menos 3 fallos reales.
4. Comparar esperado vs observado.
5. Implementar o planificar una mejora pequeña.

Done

- Tabla de modos de fallo con runbooks básicos.
-

Ejercicio 35 — Snapshot operativo y alertas básicas integradas

Ficha rápida

Campo	Valor
Tipo	producto operativo
Área	Snapshot + alertas + dashboard
Prioridad	P1
Tamaño	M
Depende de	27, 33, 34
Desbloquea	42, 44
No entra	No implementar motor complejo de reglas; no meter notificaciones externas.

¿Qué resuelve?

Une observabilidad técnica con lectura operativa visible.

Mini ejercicios

1. Incorporar stale/offline al snapshot o dashboard.
2. Mostrar estado por zona/sensor.
3. Verificar comportamiento sin datos y con backend caído.

Done

- Vista operativa coherente con la semántica definida.
-

Bloque 10 — Actuadores y control cerrado

Ejercicio 36 — Modelo de comando de actuador

Ficha rápida

Campo	Valor
Tipo	diseño actuadores
Área	ActuatorCommand / MQTT cmd
Prioridad	P2
Tamaño	M
Depende de	21, 22
Desbloquea	37
No entra	No accionar hardware real sin seguridad; no saltarse auditoría/autorización.

¿Qué resuelve?

Evita que un actuador se trate como un simple botón sin trazabilidad.

Mini ejercicios

1. Definir estados de comando.
2. Diseñar entidad `ActuatorCommand`.
3. Diseñar endpoint y payload MQTT.
4. Integrar `correlationId` y autorización.
5. Documentar qué parte es diseño y qué parte implementación real.

Done

- Modelo claro y defendible.
-

Ejercicio 37 — Timeout y confirmación de actuador

Ficha rápida

Campo	Valor
Tipo	resiliencia actuadores
Área	Timeout / confirmación / estados
Prioridad	P2
Tamaño	M
Depende de	36
Desbloquea	42, 44
No entra	No depender de éxito optimista; no dejar comandos pendientes indefinidamente.

¿Qué resuelve?

Evita comandos eternamente pendientes cuando el actuador no responde.

Mini ejercicios

1. Definir tiempo máximo de confirmación.
2. Simular comando sin `state` de vuelta.
3. Implementar transición a `TIMEOUT` o dejar mecanismo definido.
4. Reflejar el estado en auditoría/API/UI si aplica.

Done

- Caso de timeout resuelto y documentado.

Bloque 11 — Refactor arquitectónico y mantenibilidad

Ejercicio 38 — System design inverso del flujo real

Ficha rápida

Campo	Valor
Tipo	arquitectura + comunicación
Área	System design / defensa técnica
Prioridad	P1
Tamaño	M
Depende de	1, 2, 3, 11
Desbloquea	39, 40, 42
No entra	No describir arquitectura idealizada; documentar lo real y sus gaps.

¿Qué resuelve?

Te obliga a explicar la arquitectura real y no la idealizada.

Mini ejercicios

1. Dibujar el flujo real actual.
2. Marcar estado, acoplamientos y puntos de fallo.
3. Justificar edge separado, platform separada y polling antes que WebSockets.
4. Escribir 3 tradeoffs actuales y 3 mejoras futuras sin reescritura total.

Done

- Documento de defensa técnica reutilizable.
-

Ejercicio 39 — Refactor con criterio de Clean Architecture

Ficha rápida

Campo	Valor
Tipo	mantenibilidad
Área	Clean Architecture / refactor seguro
Prioridad	P2
Tamaño	M
Depende de	38
Desbloquea	41
No entra	No hacer reescrituras grandes; tocar una responsabilidad concreta.

¿Qué resuelve?

Reduce responsabilidades mal ubicadas y duplicación de conocimiento.

Mini ejercicios

1. Detectar una sola responsabilidad mal ubicada.
2. Escribir problema y diseño objetivo mínimo.
3. Asegurar tests.
4. Refactorizar sin cambiar comportamiento externo.
5. Comparar antes/después y riesgos.

Done

- Refactor pequeño, real y seguro.
-

Ejercicio 40 — ADRs de decisiones importantes

Ficha rápida

Campo	Valor
Tipo	arquitectura documental
Área	ADR / decisiones técnicas
Prioridad	P2
Tamaño	S
Depende de	18, 38, 39
Desbloquea	41, 44
No entra	No convertir ADRs en ensayos largos; una decisión por archivo.

¿Qué resuelve?

Evita que las decisiones queden perdidas en conversaciones, commits o memoria.

Mini ejercicios

1. Crear plantilla ADR.
2. Redactar ADR sobre outbox SQLite.
3. Redactar ADR sobre Node-RED como bridge.
4. Redactar ADR sobre polling antes que WebSockets.
5. Enlazar ADRs desde README o índice docs.

Done

- Al menos 3 ADRs reales.

Bloque 12 — Profesionalización y entrega

Ejercicio 41 — Pipeline mínimo de confianza

Ficha rápida

Campo	Valor
Tipo	calidad de entrega
Área	Pipeline local / smoke tests
Prioridad	P0
Tamaño	M
Depende de	2, 6, 7, 11, 15, 29
Desbloquea	44
No entra	No montar CI/CD cloud complejo; empezar por checklist/script local.

¿Qué resuelve?

Reduce miedo a romper el proyecto y convierte validación en disciplina.

Mini ejercicios

1. Definir checks imprescindibles backend.
2. Definir checks imprescindibles frontend.
3. Definir smoke test de Compose.
4. Definir smoke test de ingest y telemetry.
5. Crear script o checklist ejecutable.

Done

- Pipeline local mínimo versionado.

Ejercicio 42 — Guion de demo técnica de 10 minutos

Ficha rápida

Campo	Valor
Tipo	portfolio + venta técnica
Área	Demo técnica 10 minutos
Prioridad	P1
Tamaño	M
Depende de	31, 35, 41
Desbloquea	44
No entra	No improvisar; no enseñar piezas frágiles sin avisar limitaciones.

¿Qué resuelve?

Convierte el proyecto en algo enseñable, defendible y repetible.

Mini ejercicios

1. Definir audiencia.
2. Elegir recorrido de demo.
3. Preparar comandos exactos.
4. Ensayar y detectar puntos frágiles.
5. Crear versión corta y versión técnica.

Done

- Demo ejecutable en 10 minutos.

Ejercicio 43 — Issue templates y PR template

Ficha rápida

Campo	Valor
Tipo	proceso profesional
Área	GitHub templates / workflow
Prioridad	P2
Tamaño	S
Depende de	Ninguno
Desbloquea	Todos los ejercicios posteriores
No entra	No burocratizar; plantillas cortas y útiles.

¿Qué resuelve?

Introduce disciplina de equipo aunque el proyecto se lleve en solitario.

Mini ejercicios

1. Crear plantilla de issue de ejercicio.
2. Crear plantilla de bug.
3. Crear PR template con evidencias y riesgos.
4. Usar las plantillas en 2 o 3 ejercicios reales.

Done

- Flujo Git más profesional y repetible.

Ejercicio 44 — Checklist de release local

Ficha rápida

Campo	Valor
Tipo	release management
Área	Checklist / tag / evidencias
Prioridad	P1
Tamaño	M
Depende de	16, 35, 41, 42
Desbloquea	Demo pública o revisión profesional
No entra	No declarar release sin limitaciones conocidas; no esconder gaps.

¿Qué resuelve?

Permite declarar una versión demo con honestidad sobre lo que funciona y lo que no.

Mini ejercicios

1. Nombrar la release.

2. Crear checklist backend, edge y frontend.
3. Ejecutarlo completo.
4. Registrar limitaciones conocidas.
5. Taggear la versión si procede.

Done

- Release local verificable.

Ubicación recomendada en el repo

Ruta sugerida:

```
docs/learning/glea-nexo-roadmap-unificado-ejercicios.md
```

Enlaces recomendados:

- enlazar desde `README.md` como roadmap de aprendizaje aplicado;
- enlazar desde `agents.md` para que los agentes respeten el orden de iteración;
- crear issues usando el número de ejercicio como prefijo, por ejemplo `EX-003 offline replay controlado`.

Checklist común de cierre

Antes de marcar un ejercicio como terminado, validar:

- ☐ Hay una rama, issue o commit identificable.
- ☐ Hay evidencia reproducible.
- ☐ Hay documentación actualizada.
- ☐ Hay prueba manual o automática.
- ☐ Hay limitaciones conocidas.
- ☐ Se puede explicar en 2 minutos como `problema → decisión → tradeoff → riesgo`.
- ☐ Si afecta API, Swagger/OpenAPI está actualizado.
- ☐ Si afecta BD, migraciones y SQL están documentadas.
- ☐ Si afecta edge, existen comandos de validación MQTT/SQLite/Node-RED.
- ☐ Si afecta frontend, hay manejo de loading/error/empty y evidencia visual.

Priorización práctica recomendada

Si el objetivo es **aprender con máxima rentabilidad técnica**, el primer recorrido recomendado es:

```
1 → 2 → 3 → 4 → 5 → 6 → 11 → 13 → 15 → 19 → 23 → 24 → 27 → 28 → 29 → 30 → 31 → 33 → 34 → 41 → 42
```

Si el objetivo es **llevar Glea Nexa antes a una demo seria**, el recorrido más útil es:

```
1 → 2 → 3 → 6 → 7 → 8 → 10 → 11 → 12 → 27 → 28 → 29 → 30 → 31 → 32 → 33 → 35 → 41 → 42 → 44
```

Si el objetivo es **hardening técnico**, el recorrido más sólido es:

TO-DOs pendientes / Ideas de mejora no incluidas en ejercicios

- Blindar Node-RED timestamps para soportar sensores heterogéneos.
- Retomar en preparación de demo técnica el caso de `messageId` reutilizado con contenido distinto: evaluar si conviene comparar contenido, registrar conflicto semántico explícito o reforzar contratos/productores/Node-RED para evitar reutilización inválida.
- Evaluar si el frontend debe migrar de `ng serve` a build estático con nginx/caddy para reducir superficie de ataque y mejorar rendimiento (no crítico para MVP, sí para release seria).
- Auditar migraciones Flyway V002 vs V003: resolver inconsistencia de códigos de unidad (`M_S` vs `M_PER_S` , `VOLTS` vs `VOLT`) consolidando el catálogo en una migración de limpieza.
- Verificar que los contratos documentados en `agents.md` (ej. respuesta de ingest `{inserted, duplicates, rejected}`) estén alineados con el código real (`{total, processed, duplicates, errors, items}`).

Recomendación final de uso

No intentes completar este roadmap como si fuera una colección de deberes. Úsalo como una cartera de iteraciones pequeñas que construyen tres activos a la vez: conocimiento profundo, mejora real del sistema y material defendible para entrevistas, portfolio y trabajo profesional.

La prioridad correcta sigue siendo la misma: primero validar verdad operativa, después cerrar contratos y semántica, luego endurecer el sistema, y solo después pulir narrativa, UX y release.